

“The Game Beneath the Game: The Evolution of Cheats and Anti-Cheat Systems in Counter-Strike 2”

Author: Pablo Valiente

Supervisor: Dr. Antonio Nappa

Universidad Carlos III de Madrid

Academic Year: 2024–2025

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Objective:** To merge this journey with an academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *“How did they do it?”*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Objective:** To merge this journey with an academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Objective:** To merge this journey with an academic study by analyzing and implementing real cheats to test detection systems.

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.

Outline

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Anti-Cheat Systems Timeline

Anti-cheat systems have evolved from basic client-side tools to complex, kernel-level detectors. Each marked a significant shift in visibility, access, or enforcement.

System	Year	Mode	Milestone or Innovation
PunkBuster	2000	User	First real-time memory scan and screenshot system
VAC	2002	User	Delayed ban model to obfuscate detection methods
BattleEye	2004	Kernel	Early adoption of kernel-mode drivers
FairFight	2012	Server	Behavioral/statistical detection at server level
EAC	2018	Kernel	Real-time scanning with driver-level protection
Vanguard	2020	Kernel	Persistent always-on kernel driver (controversial)
VAC Live	2023	Hybrid	Real-time detection with low-latency enforcement

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

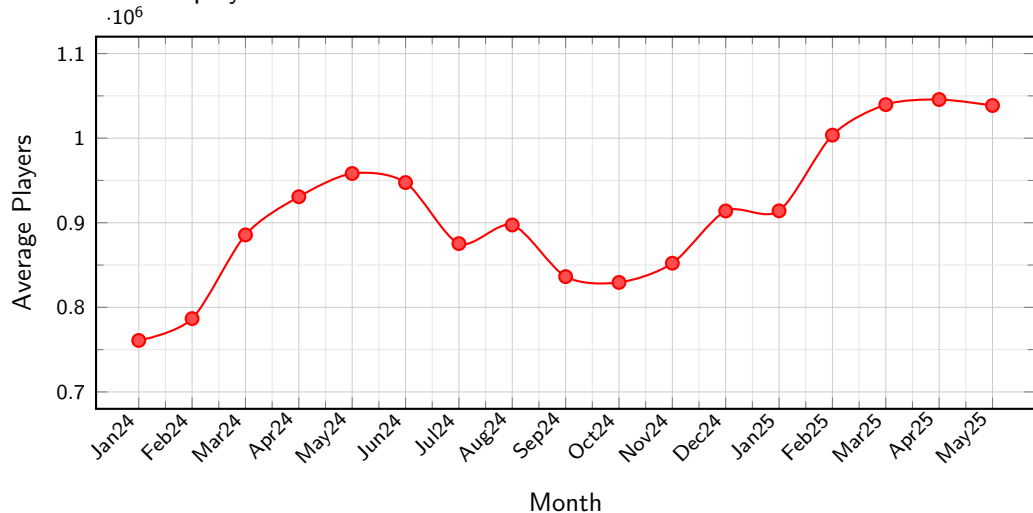
Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

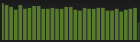
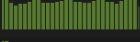
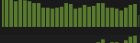
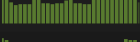
CS2 Player Base Growth (2024–2025)

To contextualize the scale of CS2's ecosystem, the following graph illustrates the evolution of its player base based on SteamCharts data.



Steam Top 10: July 7, 2025

Top Games By Current Players

	Name	Current Players	Last 30 Days	Peak Players	Hours Played
1.	Counter-Strike 2	1,222,782		1,725,815	729,749,131
2.	PUBG: BATTLEGROUNDS	581,616		778,327	226,931,487
3.	Dota 2	508,454		648,095	300,471,647
4.	Bongo Cat	169,028		192,501	110,247,318
5.	Delta Force	164,154		177,775	60,062,183
6.	Apex Legends	150,271		250,796	73,537,334
7.	Stardew Valley	133,590		156,589	47,739,319
8.	Rust	129,532		187,345	73,879,862
9.	Wallpaper Engine	120,513		138,702	61,757,771
10.	Grand Theft Auto V Legacy	90,856		112,862	53,501,676

[More](#)

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system, VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

Survey: Cheating as a Critical Threat

A 2018 global survey by Irdeto revealed the widespread negative impact of cheating in Counter-Strike:

Survey Statement	Percentage
Experienced negative impact due to cheating	60%
Would stop playing a game because of cheaters	77%
Support permanent bans for cheaters	69%
Believe developers are not doing enough	55%

- Highlights long-term damage to player trust and retention.
- Economic implications due to reduced engagement and spending.

Outline

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

DLL Injection and Manual Mapping

Cheats are commonly implemented as DLL files injected into the game process.

- **Classic Methods:** LoadLibrary and CreateRemoteThread are frequently used, but easily detected.
- **Manual Mapping:** Avoids standard API calls, skipping LoadLibrary and not registering the module.
- **Malware Parallels:** These techniques mirror malware behavior, stealthy, evasive, and persistent.

VAC Detection and Countermeasures

VAC employs both traditional and modern mechanisms to detect unauthorized modifications within the game environment.

- **Detection Vectors:** Signature scanning, analysis of suspicious thread start addresses, and detection of VMT (Virtual Method Table) hooks are among its key strategies.
- **Reversing Insight:** Inspection of `steamservice.dll` reveals that VAC dynamically streams detection modules from Valve's servers into memory at runtime.
- **Dumper Feasibility:** With moderate reverse engineering effort, it is possible to create a dumper capable of capturing and writing these modules to disk for offline analysis.

VAC Detection and Countermeasures

VAC employs both traditional and modern mechanisms to detect unauthorized modifications within the game environment.

- **Detection Vectors:** Signature scanning, analysis of suspicious thread start addresses, and detection of VMT (Virtual Method Table) hooks are among its key strategies.
- **Reversing Insight:** Inspection of `steamservice.dll` reveals that VAC dynamically streams detection modules from Valve's servers into memory at runtime.
- **Dumper Feasibility:** With moderate reverse engineering effort, it is possible to create a dumper capable of capturing and writing these modules to disk for offline analysis.

VAC Detection and Countermeasures

VAC employs both traditional and modern mechanisms to detect unauthorized modifications within the game environment.

- **Detection Vectors:** Signature scanning, analysis of suspicious thread start addresses, and detection of VMT (Virtual Method Table) hooks are among its key strategies.
- **Reversing Insight:** Inspection of `steamservice.dll` reveals that VAC dynamically streams detection modules from Valve's servers into memory at runtime.
- **Dumper Feasibility:** With moderate reverse engineering effort, it is possible to create a dumper capable of capturing and writing these modules to disk for offline analysis.

VAC and Valve Anti-Cheat Visuals



VAC Inspecting Suspicious Threads



Valve Updates Anti Cheat

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Core Role:** Alongside reverse engineering, hooking forms the backbone of our cheat architecture.
- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Core Role:** Alongside reverse engineering, hooking forms the backbone of our cheat architecture.
- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Core Role:** Alongside reverse engineering, hooking forms the backbone of our cheat architecture.
- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Hooking the Engine: Input, Rendering, and Events

Hooking allows interception of internal engine calls to alter behavior at runtime.

- **Core Role:** Alongside reverse engineering, hooking forms the backbone of our cheat architecture.
- **Initial Step:** Reverse engineer the game binary to locate key virtual functions related to input, rendering, or entity updates.
- **Hooking Layer:** Inject trampoline hooks to redirect control flow to our cheat-defined functions.
- **Limitations:** While MinHook is sufficient for prototyping, more stealth-oriented alternatives exist (e.g., Shadow Hooking, Page Guard, or VEH-based hooks).

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Entity Management:** The game uses a handle-based paging system to manage all in-game entities.
- **Cheat Integration:** Our entity list must stay in sync with the game to avoid crashes from invalid pointers.
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Entity Management:** The game uses a handle-based paging system to manage all in-game entities.
- **Cheat Integration:** Our entity list must stay in sync with the game to avoid crashes from invalid pointers.
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Entity Management:** The game uses a handle-based paging system to manage all in-game entities.
- **Cheat Integration:** Our entity list must stay in sync with the game to avoid crashes from invalid pointers.
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.

Entity List in Reclass

```
Structure dissection named structure
File View Structures Structure Options
Group 1
+ x.dll=0x19f7f00 ← your EntityList Offset

Offset-description Address Value
unnamed structure
  0000 - Pointer to instance of CGameEntitySystem ← The EntitySystem you need 7FFD03B87F00 : 0--81922800
    > 0000 - Pointer E1922800 : 0--7FFD03B87F00
    -0008 - 4 Bytes E1922808 : 0
    -000C - 4 Bytes E192280C : 0
  0010 - Pointer to Autocreated from E1360308 ← 0x10 byte offset
    > 0000 - Pointer to instance of C_World ← First Entity E1360308 : 0--308670000
    > 0008 - Pointer E1360310 : 0--7FFD03B87F00
    -0010 - 4 Bytes (Hex) E1360318 : 00000000
    -0014 - 4 Bytes E136031C : 1234567295
    -0018 - 4 Bytes E1360320 : 0
    -001C - 4 Bytes E1360324 : 0
    > 0020 - Pointer E1360328 : 0--82236025
    -0028 - 4 Bytes E1360330 : 0
    -002C - 4 Bytes E1360334 : 0
    -0030 - 4 Bytes E1360338 : 0
    -0034 - 4 Bytes E136033C : 0
    -0038 - 4 Bytes E1360340 : 0
    -003C - 4 Bytes E1360344 : 0
    > 0040 - Pointer E1360348 : 0--00000000
    -0048 - 4 Bytes E1360350 : 0
    -004C - 4 Bytes E1360354 : 0
    -0050 - 4 Bytes E1360358 : 0
    -0054 - 4 Bytes E136035C : 0
    > 0058 - Pointer E1360360 : 0--82236008
    > 0060 - Pointer E1360368 : 0--81360300
    -0068 - 4 Bytes E1360370 : 0
    -006C - 4 Bytes E1360374 : 0
    -0070 - 4 Bytes E1360378 : 0
    -0074 - 4 Bytes E136037C : 0
    > 0078 - Pointer to instance of CCSPlayerController ← Second Entity E1360380 : 0--311200000
    > 0080 - Pointer E1360388 : 0--7FFD03B87F00
    -0088 - 4 Bytes (Hex) E1360390 : 00F20001
    -008C - 4 Bytes E1360394 : 1234567295
    -0090 - 4 Bytes E1360398 : 0
    -0094 - 4 Bytes E136039C : 0
    > 0098 - Pointer E13603A0 : 0--822360207
    -00A0 - 4 Bytes E13603A8 : 0
    -00A4 - 4 Bytes E13603AC : 0
    -00A8 - 4 Bytes E13603B0 : 0
    -00AC - 4 Bytes E13603B4 : 0
    -00B0 - 4 Bytes E13603B8 : 0
    -00B4 - 4 Bytes E13603BC : 0
    > 00B8 - Pointer E13603C0 : 0--00000000
```

Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: `.vk3`).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).

Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).

Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).

Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: .vk3).
- **Material Creation:** This allowed us to mimic and reconstruct materials programmatically.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g., player entities, weapons).

Material VK3 structure

```
static constexpr char szVMatBufferMetalic[] =
    R"(<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
{
    Shader = "csgo_complex.vfx"
        F_IGNOREZ = 0
        F_DISABLE_Z_WRITE = 0
        F_DISABLE_Z_BUFFERING = 0
F_RENDER_BACKFACES = 0
    F_TRANSLUCENT = 1
    F_PAINT_VERTEX_COLORS = 1
    g_vColorTint = [1.000000, 1.000000, 1.000000, 1.000000]
    TextureNormal = resource:"materials/default/default_normal.tga"
    g_tAmbientOcclusion = resource:"materials/default/default_ao_tga_559f1ac6.vtex"
    g_tColor = resource:"materials/default/default_color_tga_72dcfbfd.vtex"
    g_tNormal = resource:"materials/default/default_normal_tga_7be61377.vtex"
})";
```

Visual Examples: Chams Materials



Visual Examples: Chams Materials



Visual Examples: Chams Materials



Visual Examples: Chams Materials



Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Rendering Strategy:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Rendering Strategy:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Rendering Strategy:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Visual Examples: Bone Matrix



Visual Examples: Skeleton



Visual Examples: Skeleton & Chams in a Real Match



Visual Examples: Skeleton in a Real Match



Outline

Motivation and Context

The Evolution of Cheating: History, Ecosystem, and Ethics

Counter-Strike 2: A Case Study in Cheating Dynamics

Code Injection and Cheat Delivery Mechanisms

Cheat Implementation: Hooking, Logic, and Features

Reflections, Implications, and Future Work

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Final Reflections

- Cheating in online games reflects a complex interplay between technical ingenuity and social dynamics.
- The continual arms race between cheat developers and anti-cheat systems mirrors broader cybersecurity challenges.
- Gaining offensive insight through cheat implementation reveals critical blind spots in detection strategies.
- Ethical, economic, and technical dimensions must be addressed together to preserve competitive integrity.

Future Work and Research Directions

- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Future Work and Research Directions

- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Future Work and Research Directions

- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore cheat detection benchmarking using controlled environments and real-time telemetry.

Thank you for your attention!

Any questions?